Robot CLI Command Tools

1. Course Content

Course Overview

- This lesson explains the use of the robot CLI (Command Line Interface) command tools. Through the command line interface, users can directly control various functions of the robot, such as chassis movement and visual recognition.
- The CLI command tools encapsulate all MCP Tool interfaces. Without relying on an AI model, you can quickly control the robot, which is convenient for functional testing and debugging.

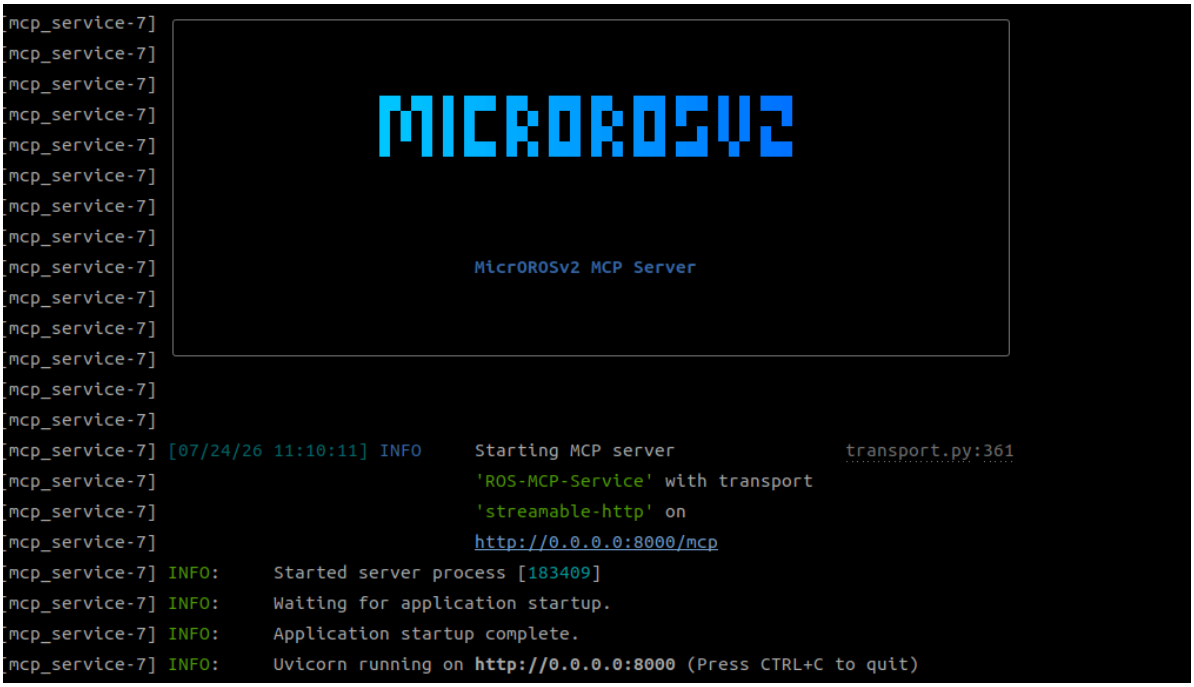
Learning Objectives

1. Understand the functional positioning and application scenarios of the robot CLI command tools
2. Become familiar with the functions, parameters, and usage format of CLI commands

> [!IMPORTANT]
>
> The demo in this lesson uses the pan-tilt camera version (PTZ) as an example; the MCP and CLI tool types may differ slightly across different vehicle versions

2. Preparation

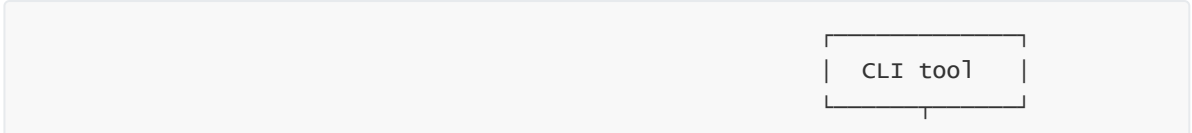
```
ros2 launch microros_v2_bringup openclaw_service.launch.py
```

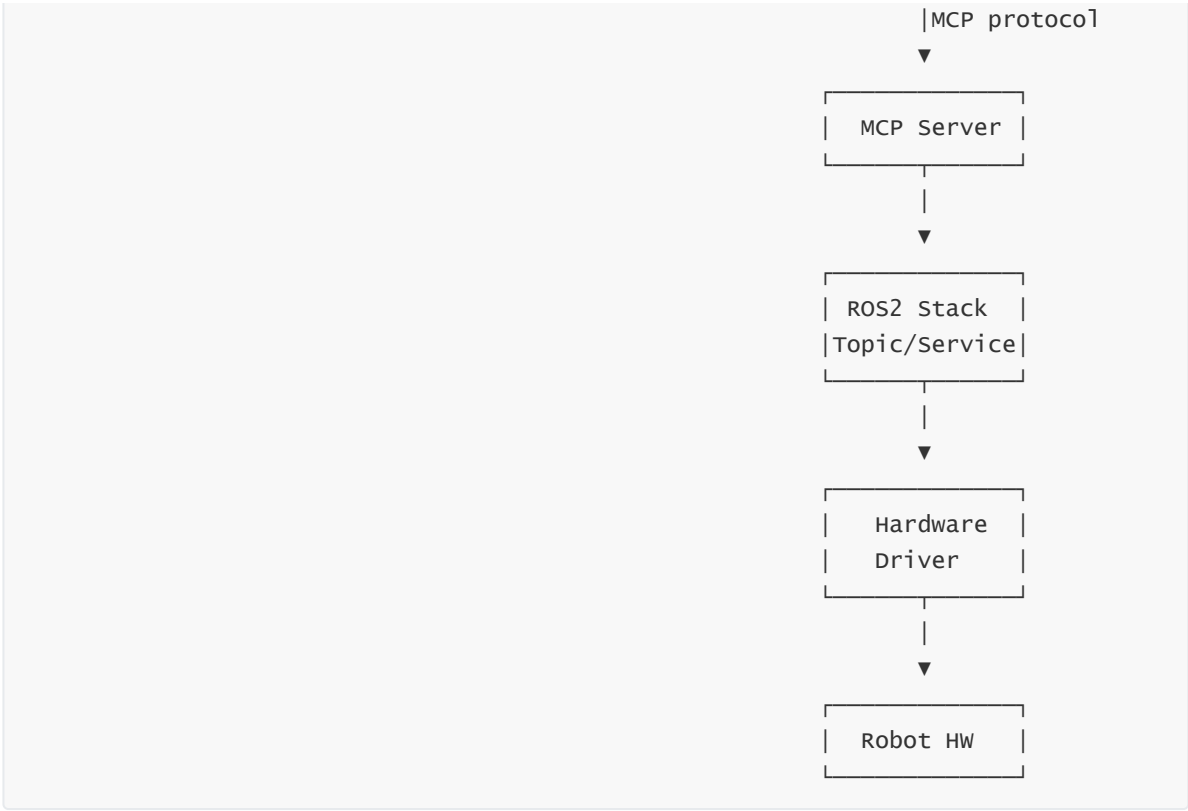


3. Introduction to CLI Tools

The robot CLI (Command Line Interface) control tool is a command-line-based robot control interface that calls the various robot control tools registered on the MCP server through the `robot_control` command. It encapsulates the robot's underlying control interfaces, converting complex ROS2 communication into concise command-line instructions. You only need to enter a command in the terminal to control the robot to perform various actions.

4. CLI Command Robot Control Communication Architecture





5. Controlling the Robot with CLI

5.1 CLI Command Summary

You can view the list of all available CLI commands with `robot_control list-tools`. The following is a summary of all currently registered CLI commands:

Command	Parameters	Function
`Move`	`--direction`, `--distance`, `--angle`	Move the robot chassis by a specified distance or rotate by a specified angle. `direction` options: `forward`, `backward`, `turn_left`, `turn_right`
`SeeWhat`	None	Capture one frame of image and return the saved path
`AdjustCameraView`	`--pitch`, `--yaw`	Adjust the camera view. `pitch` controls tilt (positive = up, negative = down), `yaw` controls pan (positive = right, negative = left); a step of 5-10 degrees is recommended
`GetBbox`	`--query`	Detect and return the bounding box (bbox) coordinates of the target object based on a text description
`TargetTrack`	`--x1-or-cmd`, `--y1`, `--x2`, `--y2`	Visual tracking of the target object. Pass `cancel` to stop tracking
`FollowLine`	`--color`	Follow a colored route on the ground. `color` options: `red`, `green`, `blue`, `yellow`
`CameraActionGroup`	`--action_name`	Execute camera preset action groups. Options: `init_camera_pose`, `nod`, `shake`
`Navigation`	`--location`	Navigate to a target location; the parameter is the location symbol in the map mapping file
`RecordMapLocation`	`--name`, `--symbol`	Save the current location to the map mapping file
`GetMapMapping`	None	Get the mapping relationship between all location names and their corresponding symbols

5.2 Tools Supported by Different Vehicle Versions

- Basic version - basic_car (no vision functions): Move, Navigation, RecordMapLocation, GetMapMapping
- Standard version - no_ptz (no pan-tilt functions): Move, Navigation, RecordMapLocation, GetMapMapping, SeeWhat, GetBbox, TargetTrack, FollowLine
- Servo pan-tilt version - ptz (supports all functions)

> [!NOTE]
>
> The demo cases in the following chapters use the servo pan-tilt version (ptz) vehicle

5.3 Command Format

All CLI commands follow a unified command format:

```
robot_control call-tool <command-name> --parameter1 <value1> --parameter2
<value2> ...
```

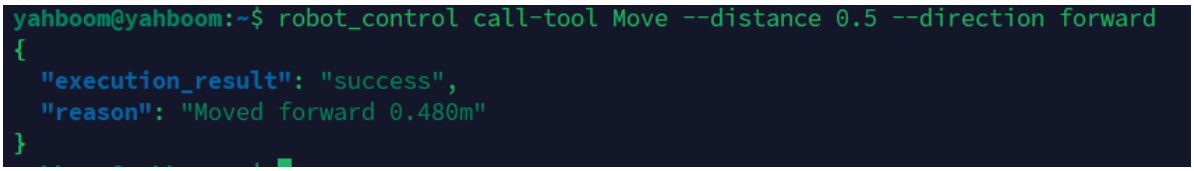
- `robot_control`: The main command of the CLI tool
- `call-tool`: The subcommand indicating a tool call
- `<command-name>`: The specific tool name registered on the MCP server, such as `Move`, `SeeWhat`, `TTS`, etc.
- `--parameter <value>`: The key-value pair of parameters passed as needed; different commands have different numbers and types of parameters

> Use `robot_control list-tools` to view the list of all available CLI commands

6. Demo Cases

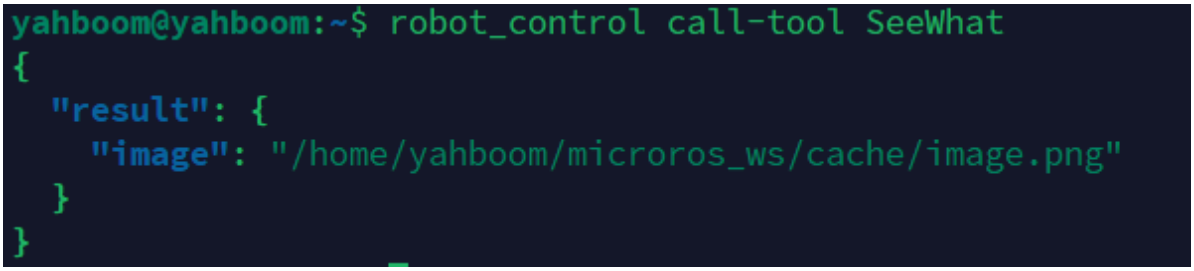
- The robot chassis moves forward 0.5 meters

```
robot_control call-tool Move --distance 0.5 --direction forward
```

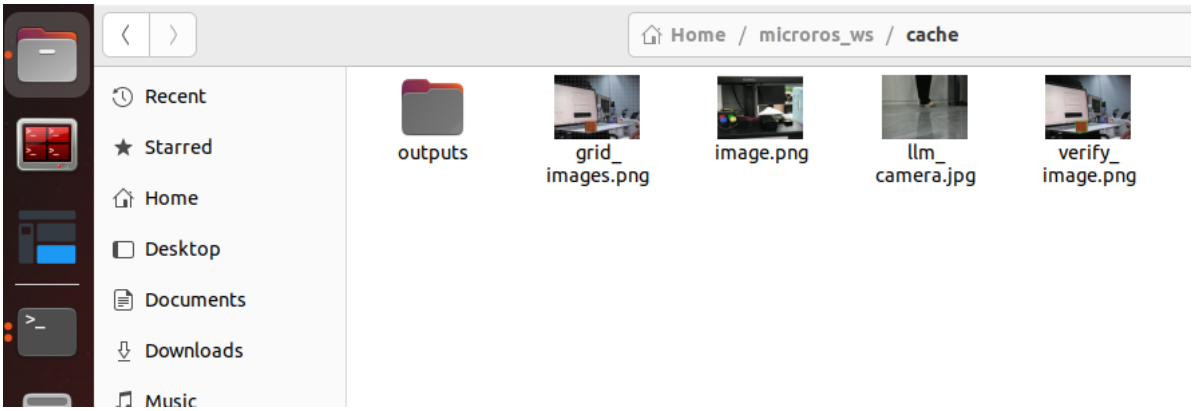


- Get an image of the environment (only for versions with a camera)

```
robot_control call-tool SeeWhat
```



> [!NOTE]
>
> When the SeeWhat tool is called, it automatically saves the image to "`$HOME/microros_ws/cache/image.png`"



7. Source Code Analysis

Source Code Path:

```
$HOME/microros_ws/src/multi_brains_pre/cli/robot_control
```

The CLI tool source code registers the various robot control functions as MCP Tools through the `register_actions()` function in `mcp_action.py`. Here, the `Move` (move the chassis by a specified distance) tool is used as an example to explain the source code:

```
@call_tool_app.command(name='Move')
async def Move(
    *,
    direction: Annotated[str, cyclopts.Parameter(help="Movement direction: 'forward', 'backward', 'turn_left', or 'turn_right'")],
    distance: Annotated[float, cyclopts.Parameter(help="Linear movement distance in meters. Only used when direction is 'forward' or 'backward'. Set to 0.0 for rotation.")] = 0.0,
    angle: Annotated[float, cyclopts.Parameter(help="Rotation angle in degrees. Only used when direction is 'turn_left' or 'turn_right'. Set to 0.0 for linear movement.")] = 0.0,
) -> None:
    '''Move the robot chassis by a specified distance or rotate by a specified angle using odometry feedback.'''
    await _call_tool('Move', {'direction': direction, 'distance': distance, 'angle': angle})
```

Code explanation:

1. **@call_tool_app.command(name='Move')**: Uses the `cyclopts` framework to register the function as a CLI subcommand; `name` defines the command name, corresponding to the tool name registered on the MCP server
2. **Function parameters**: CLI parameters are defined through the `Annotated` type annotation plus `cyclopts.Parameter`; `help` sets the parameter description. `direction` is a required parameter, while `distance` and `angle` are optional parameters with a default value of 0.0
3. **_call_tool**: A unified tool call function. Internally, it connects to the MCP server through `fastmcp.Client`, filters the parameters, sends them to the server for execution, and prints the returned result